1. insert NAME ID

- Time Complexity: O(log N)
- Space Complexity: O(log N) auxiliary stack space for recursion
- Why: Because AVL tree self-balance itself, the height is bounded by O(log N). Finding the correct insertion spot, updating heights, and performing at most one rotation (or double rotation) per ancestor along the path takes logarithmic time.

2. remove ID

- Time Complexity: O(log N)
- Space Complexity: O(log N) auxiliary stack space
- Why: Finding the node requires traversing down the tree height. Finding the inorder successor (in the 2-children case) takes time proportional to the height of the right subtree (O(log N)). The recursion takes O(log N).

3. search ID

- Time Complexity: O(log N)
- Space Complexity: O(log N) auxiliary stack space
- Why: Since IDs are used as the ordering keys for the AVL tree structure, searching by ID acts like a standard binary search tree lookup down the height of the tree.

4. search NAME

- Time Complexity: O(N)
- Space Complexity: O(N) for storing results and recursion stack
- Why: Names are *not* the ordering keys of the AVL tree. Therefore, a search by name cannot use the tree's sorting property and requires a full traversal (visiting every node) to check all matches.

5. printInorder

- Time Complexity: O(N)
- Space Complexity: O(N) to store names in the vector plus O(log N) recursion stack
- Why: Every node in the tree must be visited once to collect and display the sorted sequence.

6. printPreorder

- Time Complexity: O(N)
- Space Complexity: O(N) to store names in the vector plus O(log N) recursion stack
- Why: Similar to in-order, a pre-order traversal visits every node exactly once.

7. printPostorder

- Time Complexity: O(N)
- Space Complexity: O(N) to store names in the vector plus O(log N) recursion stack
- Why: Every node is visited once during the post-order traversal sequence.

8. printLevelCount

- Time Complexity: O(1) or O(N) depending on implementation (in a standard AVL node structure tracking height, retrieving root height is O(1))
- Space Complexity: O(1)
- Why: Since every node maintains an updated height field, fetching the tree's level count is an instantaneous lookup.

9. removeInorder N

- Time Complexity: O(N)
- Space Complexity: O(N) to store pointers in the intermediate vector
- Why: gathered all node pointers via an in-order traversal into a vector to access the N-th element by index in O(1) time, followed by an O(log N) AVL removal. The traversal dominates the time complexity at O(N).